```
# IST 736 Final Project - Fake News Classification Pipeline

# Library Imports and Setup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix
from wordcloud import WordCloud
import spacy


from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd

# File paths
train_path = '/content/drive/MyDrive/train (2).csv'
test_path = '/content/drive/MyDrive/test (1).csv'
eval_path = '/content/drive/MyDrive/evaluation.csv'

# Load CSVs with appropriate parsing (handling bad lines and semicolon delimiters)
train_df = pd.read_csv(train_path, sep=';', engine='python', on_bad_lines='skip')
test_df = pd.read_csv(test_path, sep=';', engine='python', on_bad_lines='skip')
eval_data_df = pd.read_csv(eval_path, sep=';', engine='python', on_bad_lines='skip')

# Drop rows with missing values in critical columns
train_df.dropna(subset=['title', 'text', 'label'], inplace=True)
test_df.dropna(subset=['title', 'text', 'label'], inplace=True)
eval_data_df.dropna(subset=['title', 'text', 'label'], inplace=True)

# Combine Title and Text into one 'Content' column
for df in [train_df, test_df, eval_data_df]:
    df['Content'] = df['title'].astype(str) + " " + df['text'].astype(str)
    df['label'] = df['label'].map({0: 'FAKE', 1: 'REAL'})

# Preview the cleaned DataFrames
print(f"Train Set: {train_df.shape}\n", train_df.head(3), "\n")
print(f"Test Set: {test_df.shape}\n", test_df.head(3), "\n")
print(f"Eval Set: {eval_data_df.shape}\n", eval_data_df.head(3))
```

```
Train Set: (24353, 5)
    Unnamed: 0                                           title  \
0           0  Palestinians switch off Christmas lights in Be...
1           1  China says Trump call with Taiwan president wo...
2           2   FAIL! The Trump Organization’s Credit Score W...

                                                text label  \
0  RAMALLAH, West Bank (Reuters) - Palestinians s...  REAL
1  BEIJING (Reuters) - U.S. President-elect Donal...  REAL
2  While the controversy over Trump s personal ta...  FAKE

                                             Content
0  Palestinians switch off Christmas lights in Be...
1  China says Trump call with Taiwan president wo...
2   FAIL! The Trump Organization’s Credit Score W...

Test Set: (8117, 5)
    Unnamed: 0                                           title  \
0           0  Live from New York, it's a Trump-Clinton remat...
1           1  Catalan separatists to lose majority in tight ...
2           2  North Carolina governor concedes election to D...

                                                text label  \
0  NEW YORK (Reuters) - Veteran actor and frequen...  REAL
1  BARCELONA (Reuters) - Catalonia s independence...  REAL
2  WINSTON-SALEM, N.C. (Reuters) - North Carolina...  REAL

                                             Content
0  Live from New York, it's a Trump-Clinton remat...
```

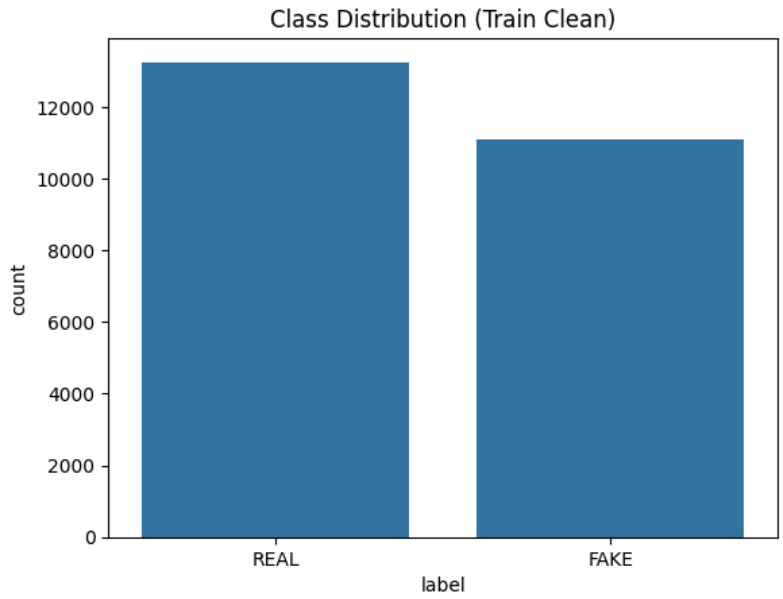
```
1  Catalan separatists to lose majority in tight ...
2  North Carolina governor concedes election to D...

Eval Set: (8117, 5)
    Unnamed: 0                                           title  \
0           0  Sanders back in U.S. Senate, blasts 'coloniali...
1           1  Kremlin: Syria peoples' congress being 'active...
2           2   Oregon Cop Convicted Of Shattering Biker’s Co...

                                                 text label  \
0  WASHINGTON (Reuters) - Democratic U.S. preside...  REAL
1  MOSCOW (Reuters) - A proposal to convene a con...  REAL
2  In a baffling fit of rage, an Oregon State Pol...  FAKE

                                            Content
0  Sanders back in U.S. Senate, blasts 'coloniali...
1  Kremlin: Syria peoples' congress being 'active...
2   Oregon Cop Convicted Of Shattering Biker’s Co...
```

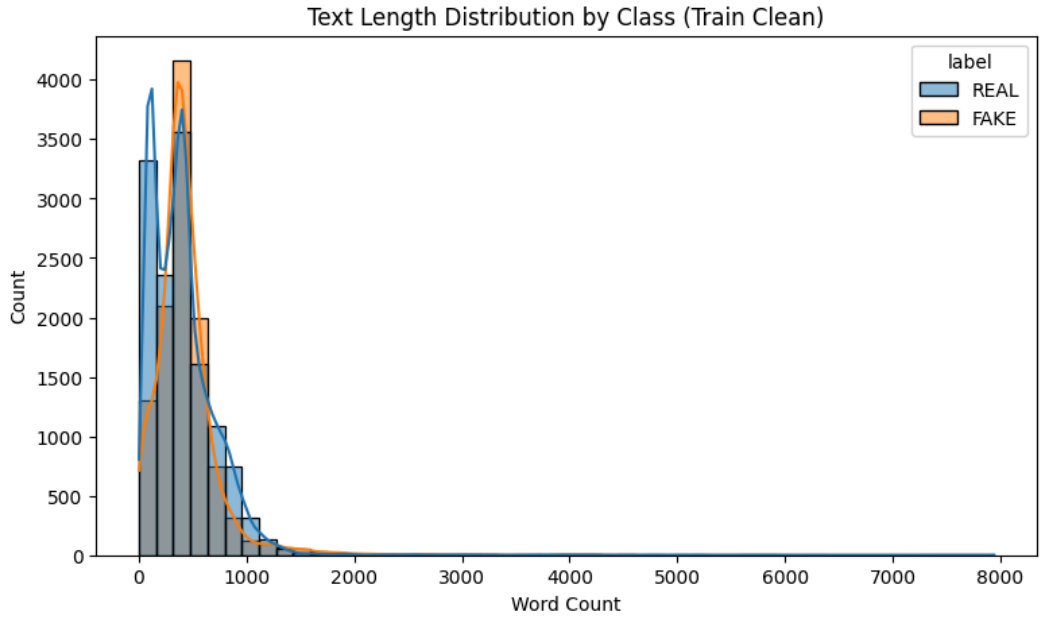
```
df_train_clean = pd.read_csv(train_path, sep=';', engine='python', on_bad_lines='skip')
df_test_clean  = pd.read_csv(test_path, sep=';', engine='python', on_bad_lines='skip')
df_eval_clean  = pd.read_csv(eval_path, sep=';', engine='python', on_bad_lines='skip')

# Review data
print("Train Data Label Distribution:")
print(df_train_clean['label'].value_counts())

print("\nTest Data Label Distribution:")
print(df_test_clean['label'].value_counts())

print("\nEvaluation Data Label Distribution:")
print(df_eval_clean['label'].value_counts())
```

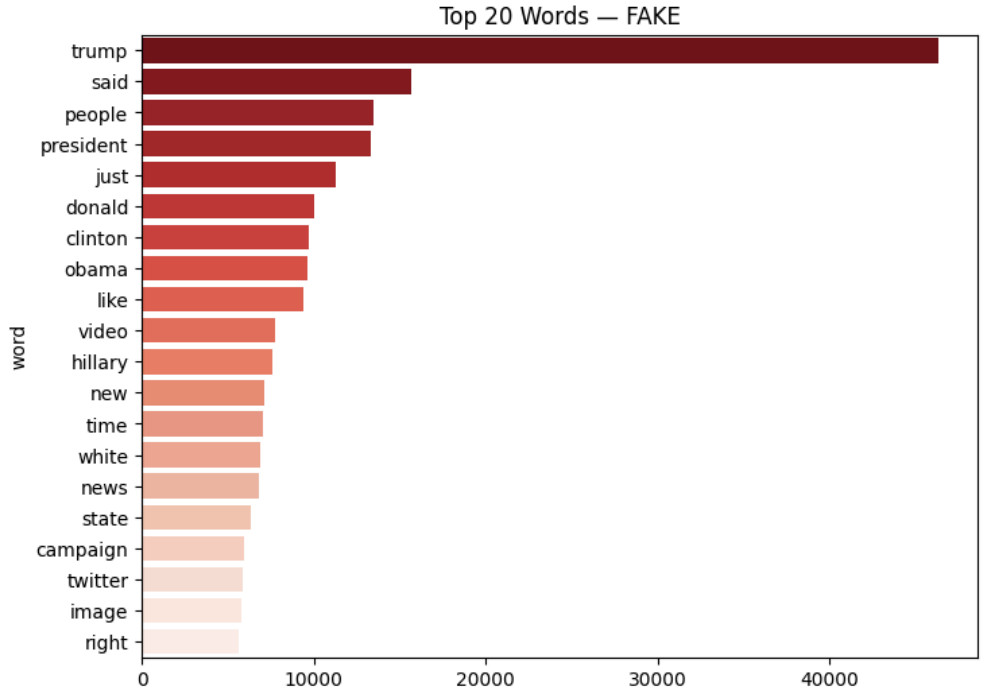
```
Train Data Label Distribution:
label
1    13246
0    11107
Name: count, dtype: int64

Test Data Label Distribution:
label
1    4364
0    3753
Name: count, dtype: int64

Evaluation Data Label Distribution:
label
1    4314
0    3803
Name: count, dtype: int64
```

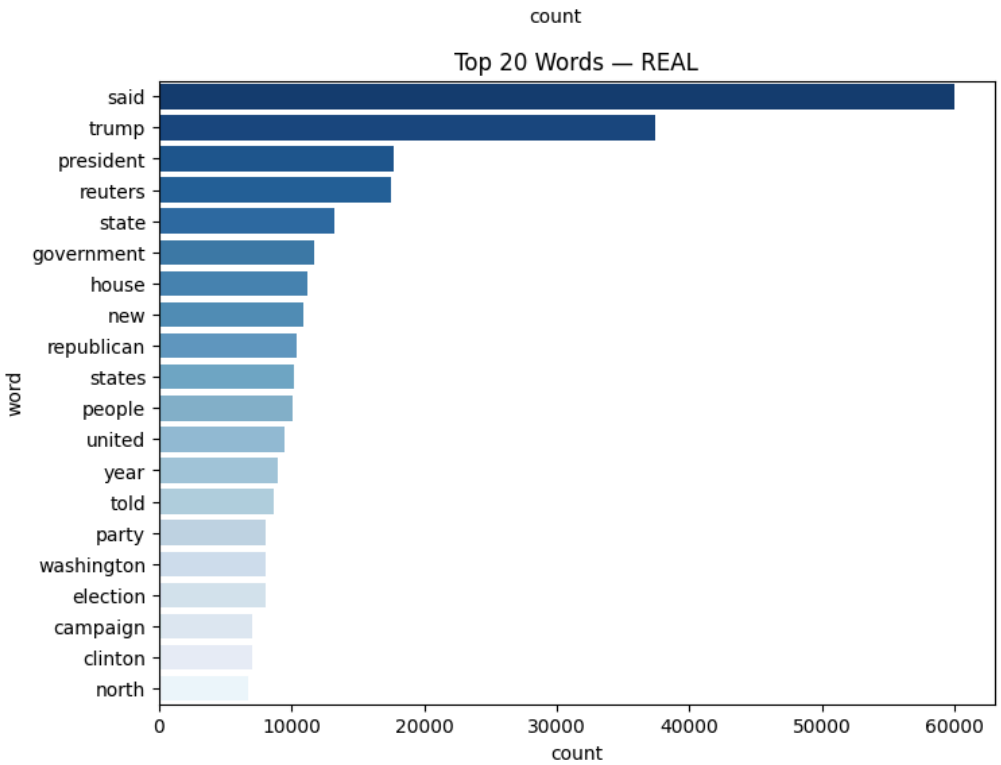
```
# EDA: Class Distribution and Document Length
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer

# Class distribution
sns.countplot(data=train_df, x='label')
plt.title('Class Distribution (Train Clean)')
plt.show()

# Document length (in words)
train_df['word_count'] = train_df['Content'].astype(str).str.split().apply(len)

plt.figure(figsize=(9,5))
sns.histplot(data=train_df, x='word_count', hue='label', bins=50, kde=True)
plt.title('Text Length Distribution by Class (Train Clean)')
plt.xlabel('Word Count')
plt.show()

# Top unigrams by class
def top_words(corpus, n=20):
    vec = CountVectorizer(stop_words='english')
    X = vec.fit_transform(corpus)
    counts = np.asarray(X.sum(axis=0)).ravel()
    vocab = np.array(vec.get_feature_names_out())
    order = counts.argsort()[::-1][:n]
    return pd.DataFrame({'word': vocab[order], 'count': counts[order]})
```

```
# Replace NaN values with empty strings before passing to top_words
top_fake = top_words(train_df[train_df['label']=='FAKE']['Content'].fillna(''), 20)
top_real = top_words(train_df[train_df['label']=='REAL']['Content'].fillna(''), 20)

plt.figure(figsize=(8,6))
sns.barplot(data=top_fake, x='count', y='word', palette='Reds_r', hue='word', legend=False)
plt.title('Top 20 Words — FAKE')
plt.show()

plt.figure(figsize=(8,6))
sns.barplot(data=top_real, x='count', y='word', palette='Blues_r', hue='word', legend=False)
plt.title('Top 20 Words — REAL')
plt.show()
```


Class Distribution (Train Clean)
count
0
2000
4000
6000
8000
10000
12000
REAL
FAKE
label


Text Length Distribution by Class (Train Clean)
label
REAL
FAKE
Count
0
500
1000
1500
2000
2500
3000
3500
4000
0
1000
2000
3000
4000
5000
6000
7000
8000
Word Count


Top 20 Words — FAKE
word
trump
said
people
president
just
donald
clinton
obama
like
video
hillary
new
time
white
news
state
campaign
twitter
image
right
0
10000
20000
30000
40000




```
# Domain stopwords via n-gram frequency

# N-gram Frequency Analysis and Custom Stopwords ===
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, CountVectorizer

# Top-n unigrams/bigrams with filtering
def get_top_words(corpus, n=50, ngram_range=(1,2), min_df=2):
    vec = CountVectorizer(
        stop_words='english',
        token_pattern=r'(?u)\b[a-zA-Z]{2,}\b',
        ngram_range=ngram_range,
        min_df=min_df
    )
    X = vec.fit_transform(corpus)
    total_counts = np.asarray(X.sum(axis=0)).ravel()
    words = vec.get_feature_names_out()
    word_freq = list(zip(words, total_counts))
    return sorted(word_freq, key=lambda x: x[1], reverse=True)[:n]

# Create corpora
fake_corpus = train_df.loc[train_df['label'] == 'FAKE', 'Content'].astype(str)
real_corpus = train_df.loc[train_df['label'] == 'REAL', 'Content'].astype(str)

# Get top words
N_TOP = 50
NGRAMS = (1,2)
top_fake_pairs = get_top_words(fake_corpus, n=N_TOP, ngram_range=NGRAMS, min_df=2)
top_real_pairs = get_top_words(real_corpus, n=N_TOP, ngram_range=NGRAMS, min_df=2)

top_fake = [w for w, _ in top_fake_pairs]
top_real = [w for w, _ in top_real_pairs]

# Tabular view
top_words_df = pd.DataFrame({
    'FAKE_top': top_fake,
    'REAL_top': top_real
})
display(top_words_df)

# Words common across both labels
common_words = set(top_fake) & set(top_real)
print("Common words across FAKE and REAL (consider as domain stopwords):")
print(sorted(common_words))

# Combine English + custom domain stopwords
```

```
custom_stopwords = ENGLISH_STOP_WORDS.union(common_words)

print(f"English stopwords: {len(ENGLISH_STOP_WORDS)}")
print(f"Common domain words added: {len(common_words)}")
print(f"Total custom stopwords: {len(custom_stopwords)}")
```

	FAKE_top	REAL_top
0	trump	said
1	said	trump
2	people	president
3	president	reuters
4	just	state
5	donald	government
6	clinton	house
7	obama	new
8	like	republican
9	donald trump	states
10	video	people
11	hillary	united
12	new	year
13	time	told
14	white	party
15	news	washington
16	state	election
17	campaign	united states
18	twitter	campaign
19	image	clinton
20	right	north
21	house	donald
22	know	white
23	media	obama
24	american	donald trump
25	america	percent
26	republican	security
27	going	court
28	don	senate
29	featured	white house
30	states	country
31	year	tax
32	make	china
33	did	officials
34	election	russia
35	say	minister
36	featured image	administration
37	way	week
38	think	law
39	country	democratic
40	republicans	national
41	years	presidential
42	government	foreign
43	told	years
44	women	military

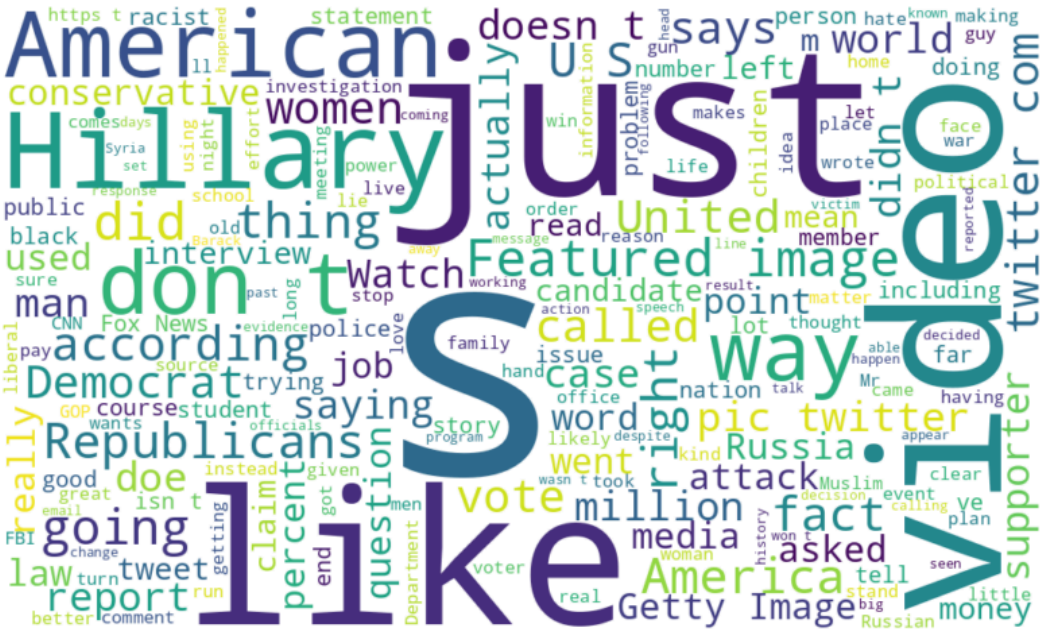
45	party	tuesday
46	want	including
47	police	political
48	watch	korea
49	hillary clinton	time

```
Common words across FAKE and REAL (consider as domain stopwords):
['campaign', 'clinton', 'country', 'donald', 'donald trump', 'election', 'government', 'house', 'new', 'obama', 'party', 'people', 'pr
English stopwords: 318
Common domain words added: 23
Total custom stopwords: 341
```

Next steps: Generate code with `top_words_df` View recommended plots New interactive sheet

```
# 3. Stopword List Construction
domain_words = {
    'campaign','clinton','country','donald','election','government','house','new','obama',
    'party','people','president','republican','said','state','states','time','told','trump','white','year','years'
}
additional_stopwords = {
    'make','use','want','need','ask','think','believe','know','try','say','told','help','work','follow',
    'look','support','include','continue','come','day','week','month','year','today','yesterday','monday',
    'tuesday','wednesday','thursday','friday','saturday','sunday','group','people','official','leader',
    'administration','authority','congress','court','business'
}
custom_stopwords = ENGLISH_STOP_WORDS.union(domain_words).union(additional_stopwords)


import spacy
nlp = spacy.load("en_core_web_sm")

def spacy_analyzer_factory(stoplist):
    def analyzer(text):
        doc = nlp(text)
        return [
            token.lemma_.lower()
            for token in doc
            if not token.is_punct
            and not token.is_space
            and token.lemma_.lower() not in stoplist
        ]
    return analyzer



analyzer = spacy_analyzer_factory(custom_stopwords)

vectorizer = TfidfVectorizer(
    analyzer=analyzer,
    max_df=0.7

)



# 5. Word Cloud Generation
def plot_wordcloud(text, title):
    wc = WordCloud(width=1000, height=600, stopwords=custom_stopwords, background_color="white", max_words=200).generate(text)
    plt.figure(figsize=(12, 6))
    plt.imshow(wc, interpolation="bilinear")
    plt.axis("off")
    plt.title(title)
    plt.show()

fake_text = " ".join(train_df[train_df["label"] == "FAKE"]["Content"].astype(str))
real_text = " ".join(train_df[train_df["label"] == "REAL"]["Content"].astype(str))
plot_wordcloud(fake_text, "Word Cloud - FAKE News")
plot_wordcloud(real_text, "Word Cloud - REAL News")
```

Word Cloud - FAKE News

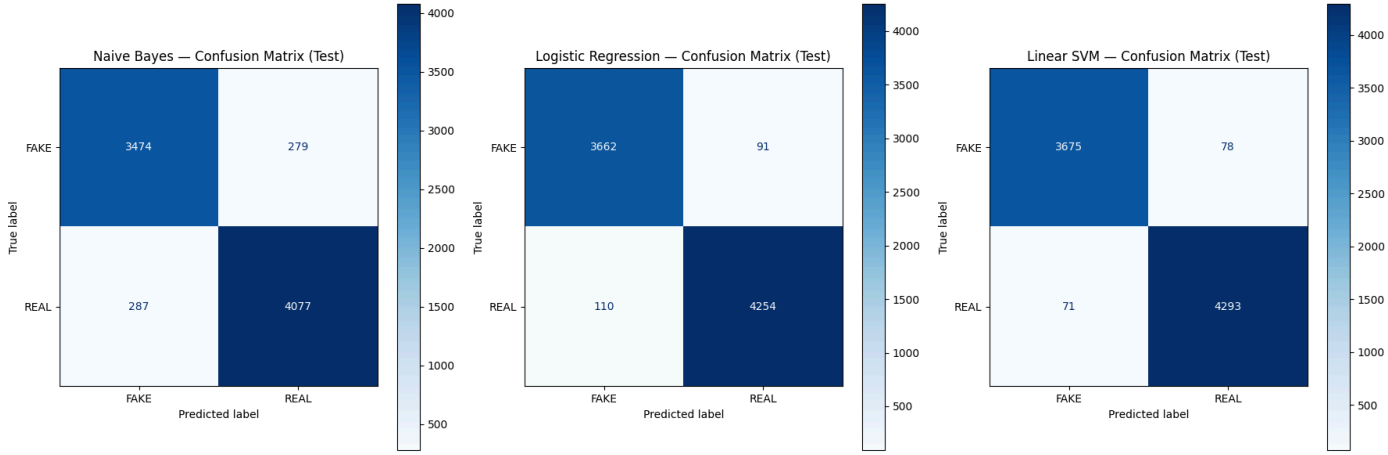


Word Cloud - REAL News

```
spacy_analyzer = spacy_analyzer_factory(custom_stopwords)

tfidf_base = TfidfVectorizer(
    analyzer=spacy_analyzer,   # if using your spaCy lemmatization pipeline
    max_df=0.7,
)


from sklearn.pipeline import Pipeline
from sklearn.metrics import ConfusionMatrixDisplay, classification_report
from IPython.display import display
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
import matplotlib.pyplot as plt

X_train, y_train = train_df['Content'], train_df['label']
X_test, y_test = test_df['Content'], test_df['label']

# Check if eval_data_df exists and has Content/label columns before using it
eval_available = 'eval_data_df' in locals() and 'Content' in eval_data_df.columns and 'label' in eval_data_df.columns
```

```
if eval_available:
    X_eval, y_eval = eval_data_df['Content'], eval_data_df['label']

print(f"Train size: {X_train.shape[0]}, Test size: {X_test.shape[0]}, Eval available: {eval_available}")

# Model definitions
models = {
    "Naive Bayes": MultinomialNB(),
    "Logistic Regression": LogisticRegression(max_iter=300, class_weight='balanced', solver='saga', n_jobs=-1),
    "Linear SVM": LinearSVC(class_weight='balanced', max_iter=1000, random_state=42)
}

# Store results
results = {}

# Define and fit tf-idf vectorizer BEFORE the loop
tfidf = TfidfVectorizer(
    analyzer=spacy_analyzer,  # Custom analyzer with tokenization, lemmatization, stopword removal
    max_df=0.7,
    min_df=2,
    max_features=10000  # Limit features for faster computation
)

# Fit the vectorizer on the training data
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)


# Iterate over models
for name, model in models.items():

    print(f"\nTraining {name}...")
    # Fit the model on the transformed training data
    model.fit(X_train_tfidf, y_train)
    # Make predictions on the transformed test data
    preds = model.predict(X_test_tfidf)

    # Evaluate and store the results
    report = classification_report(
        y_test, preds,
        labels=['FAKE', 'REAL'],
        target_names=['FAKE', 'REAL'],
        output_dict=True,
        zero_division=0
    )
    results[name] = report['weighted avg']  # precision/recall/f1 (weighted)

# Plot all confusion matrices
fig, axes = plt.subplots(1, len(models), figsize=(18, 6)) # Increased figure size for better visibility

for i, (name, model) in enumerate(models.items()):
    # Make predictions on the transformed test data for plotting
    preds = model.predict(X_test_tfidf)
    ConfusionMatrixDisplay.from_predictions(
        y_test, preds,
        labels=['FAKE', 'REAL'],
        display_labels=['FAKE', 'REAL'],
        cmap='Blues',
        values_format='d',
        ax=axes[i]
    )
    axes[i].set_title(f"{name} — Confusion Matrix (Test)")

plt.tight_layout()
plt.show()

# Display results (precision, recall, f1)
print("\nModel comparison on TEST (weighted avg):")
results_df = pd.DataFrame(results).T[['precision', 'recall', 'f1-score']]
results_df = results_df.sort_values('f1-score', ascending=False)
display(results_df)
```

```
Train size: 24353, Test size: 8117, Eval available: True

Training Naive Bayes...

Training Logistic Regression...

Training Linear SVM...
```



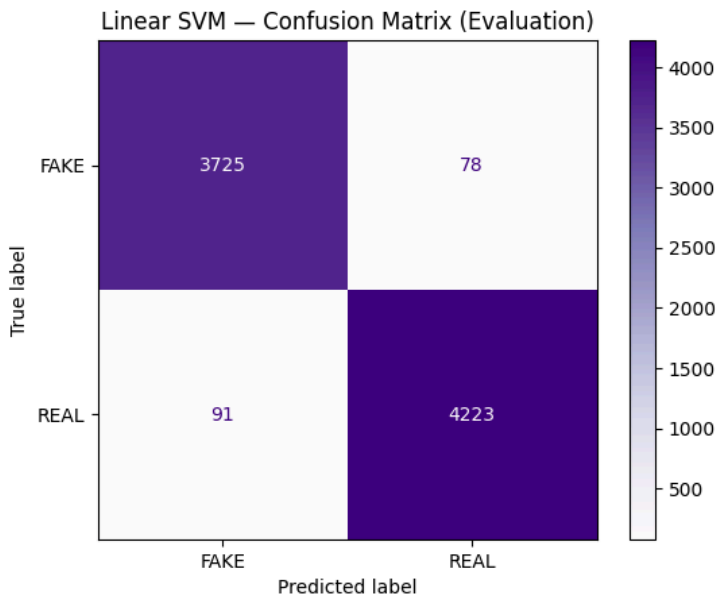

```
Model comparison on TEST (weighted avg):
```

	precision	recall	f1-score
Linear SVM	0.981643	0.981643	0.981642
Logistic Regression	0.975256	0.975237	0.975241
Naive Bayes	0.930282	0.930270	0.930275

Next steps: Generate code with `results_df` | View recommended plots | New interactive sheet

```
if eval_available:
    best_name = results_df.index[0]
    print(f"\n Evaluating best model on EVALUATION set: {best_name}")

    # Refit the best model on TRAIN (same settings)
    best_model = models[best_name]
    best_pipe = Pipeline([
        ('tfidf', TfidfVectorizer(analyzer=spacy_analyzer, max_df=0.7, min_df=2)),
        ('clf', best_model)
    ])
    best_pipe.fit(X_train, y_train)
    eval_preds = best_pipe.predict(X_eval)

    print("\nClassification Report (EVAL):")
    print(classification_report(
        y_eval, eval_preds,
        labels=['FAKE', 'REAL'],
        target_names=['FAKE', 'REAL'],
        zero_division=0
    ))

    ConfusionMatrixDisplay.from_predictions(
        y_eval, eval_preds,
        labels=['FAKE', 'REAL'],
        display_labels=['FAKE', 'REAL'],
        cmap='Purples',
        values_format='d'
    )
    plt.title(f"{best_name} — Confusion Matrix (Evaluation)")
    plt.show()
```

```
Evaluating best model on EVALUATION set: Linear SVM

Classification Report (EVAL):
              precision    recall  f1-score   support

        FAKE       0.98      0.98      0.98      3803
        REAL       0.98      0.98      0.98      4314

    accuracy                           0.98      8117
   macro avg       0.98      0.98      0.98      8117
weighted avg       0.98      0.98      0.98      8117
```




```
# ===== Train & Export: self-contained, no external imports needed =====
# (1) deps (safe to re-run)
get_ipython().system('pip -q install "spacy<4" scikit-learn joblib')
get_ipython().system('python -m spacy download en_core_web_sm')

import spacy
import joblib
import numpy as np
import pandas as pd

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

# ---------- Inline spaCy preprocessor (pickle-safe) ----------
class SpacyPreprocessor(BaseEstimator, TransformerMixin):
    """
    Turns raw text into a cleaned string of lemmas, removing tokens
    in the provided stoplist. Loads spaCy model lazily in fit().
    """
    def __init__(self, stoplist=None, spacy_model="en_core_web_sm"):
        self.stoplist = set(stoplist or [])
        self.spacy_model = spacy_model
        self._nlp = None

    def fit(self, X, y=None):
        if self._nlp is None:
            self._nlp = spacy.load(self.spacy_model, disable=["ner", "parser", "textcat"])
        return self

    def transform(self, X):
        docs = self._nlp.pipe(X, batch_size=256)
        cleaned = []
        for doc in docs:
            toks = [
                t.lemma_.lower()
                for t in doc
                if not t.is_punct and not t.is_space
```

```
                and t.lemma_.lower() not in self.stoplist
            ]
            cleaned.append(" ".join(toks))
        return cleaned

# ---------- Helper to derive X, y from an existing DataFrame ----------
def ensure_content_and_labels(df):
    # Build text
    if 'Content' in df.columns:
        X = df['Content'].astype(str)
    else:
        title = df.get('title', '').fillna('').astype(str)
        text  = df.get('text',  '').fillna('').astype(str)
        if isinstance(title, str) and isinstance(text, str):
            raise ValueError("Need 'Content' OR ('title' and 'text') columns.")
        X = (title + ' ' + text).str.strip()

    # Build labels (0/1 or 'FAKE'/'REAL')
    label_col = 'label' if 'label' in df.columns else ('Label' if 'Label' in df.columns else None)
    if label_col is None:
        raise ValueError("Need a label column: 'label' (0/1) or 'Label' ('FAKE'/'REAL').")

    y_raw = df[label_col]
    # Keep labels as strings 'FAKE' or 'REAL' if that's how they are in the DataFrame
    if set(y_raw.dropna().unique()) <= {0, 1}:
        y = y_raw.map({0: 'FAKE', 1: 'REAL'}) # Map to strings for consistency
    else:
        y = y_raw.copy() # Assume they are already 'FAKE'/'REAL' strings
        if set(y.dropna().unique()) not in [{'FAKE', 'REAL'}, {'FAKE'}, {'REAL'}, set()]:
             raise ValueError("Labels must be 0/1 or 'FAKE'/'REAL'.")
    return X.fillna(''), y

# ---------- Get training data ----------
# If you already defined X_train / y_train earlier, we'll use them as-is.
try:
    _ = X_train  # noqa
    _ = y_train  # noqa
    print("Using existing X_train/y_train in memory.")
except NameError:
    # Otherwise derive from train_df (expects you've loaded it already)
    try:
        # Ensure labels are strings 'FAKE'/'REAL' here
        X_train, y_train = ensure_content_and_labels(train_df)
        print(f"Derived X_train/y_train from train_df. Train size: {len(X_train)}")
    except Exception as e:
        raise RuntimeError(
            "Couldn't find X_train/y_train or a valid train_df. "
            "Make sure you've loaded your training CSV into 'train_df'."
        ) from e

# ---------- Stopwords ----------
# Use your custom stopword set if it exists; otherwise fall back to empty set.
try:
    custom_stopwords
except NameError:
    custom_stopwords = set()
    print("Note: 'custom_stopwords' not found — proceeding without extra domain stopwords.")

# ---------- Build & fit the pipeline ----------
pipe = Pipeline([
    ("prep", SpacyPreprocessor(stoplist=custom_stopwords, spacy_model="en_core_web_sm")),
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=2, max_df=0.7)),
    ("clf", LinearSVC(class_weight="balanced", C=1.0))
])

print("Fitting model…")
pipe.fit(X_train, y_train)
joblib.dump(pipe, "fake_news_pipeline.joblib")
print("✅ Saved: fake_news_pipeline.joblib")

# ---------- Quick sanity check ----------
sample_text = "Officials confirm new policy update from the White House."
# Directly use the string output from predict
pred_label = pipe.predict([sample_text])[0]
print(f"Sanity check → '{sample_text[:45]}…' → {pred_label}")
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 42.5 MB/s eta 0:00:00
✔ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Using existing X_train/y_train in memory.
Fitting model…
✅ Saved: fake_news_pipeline.joblib
Sanity check → 'Officials confirm new policy update from the …' → FAKE
```

```
get_ipython().system('pip install streamlit "spacy<4" scikit-learn joblib')
get_ipython().system('python -m spacy download en_core_web_sm')
get_ipython().system('streamlit run streamlit_app.py &')
```

```
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (2.0.11)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (3.0.10)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (8.3.6)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (2.5.1)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.1.0 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (0.4.1)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (0.17.3)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (4.67.1)
Requirement already satisfied: pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (2.11.7
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy<4) (75.2.0)
Requirement already satisfied: langcodes<4.0.0,>=3.2.0 in /usr/local/lib/python3.12/dist-packages (from spacy<4) (3.5.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: jsonschema>=3.0 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<6,>=4.0->strea
Requirement already satisfied: narwhals>=1.14.2 in /usr/local/lib/python3.12/dist-packages (from altair!=5.4.0,!=5.4.1,<6,>=4.0->stre
Requirement already satisfied: gitdb<5,>=4.0.1 in /usr/local/lib/python3.12/dist-packages (from gitpython!=3.1.19,<4,>=3.0.7->streaml
Requirement already satisfied: language-data>=1.2 in /usr/local/lib/python3.12/dist-packages (from langcodes<4.0.0,>=3.2.0->spacy<4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3,>=1.4.0->streamlit) (2025.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->spacy<4) (3.0.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (2.5
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.27->streamlit) (202
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.